

Id be able to select from a subset selected at the namespace level. Tracking of these features can be found in this Epic.

We will build out the capabilities for model switching in Duo Code Review, Vulnerability analysis and other features. In addition to this we also want to allow self hosted customers to bring their own models.

> Note: This design does not handle the scenario where we might want to pick from different recommended models based on the user query. For example we might want to pick Google Gemini when the context length becomes very large or Claude Sonnet when its a coding related question. Dynamic model switching would have to be covered in a separate blueprint.

New Design (End State)

This represents the end state and is not broken down by phase. Every phase would deliver a section of this architecture.

Data Model

The data model is designed to provide a structured way for organizations to manage AI feature settings at different levels. It allows namespace administrators to define default AI models for various features (e.g., Code Suggestions, Chat, etc.), while also enabling individual users to override these defaults with their preferred AI models.

The objective is to balance administrative control with user flexibility, ensuring that AI-powered features align with both organizational policies and personal preferences.

mermaid

erDiagram

```
NAMESPACES {
    bigint id PK
    bigint parentid FK
    bigint ownerid FK
    varchar name
    varchar path
    bigint organizationid FK
}
```

```
NAMESPACEFEATURESETTINGS {
    bigint id PK
    bigint namespaceid FK
    timestamp createdat
    timestamp updatedat
    smallint feature
    varchar offeredmodelref
    varchar offeredmodelname
}
```

```
NAMESPACES ||--o{ NAMESPACEFEATURESETTINGS : "has many"
```

```

USERS {
  bigint id PK
  varchar username UNIQUE
  varchar email UNIQUE
}

```

```

USERAIMODELPREFERENCES["USERAIMODELPREFERENCES (Future Iteration)"] {
  bigint id PK
  bigint userid FK
  varchar feature
  varchar favouritemodel
}

```

USERS ||--o{ USERAIMODELPREFERENCES : "has many"

Entities and Relationships

1. NAMESPACES

- Represents groups/subgroups in GitLab.
- Existing

2. NAMESPACEFEATURESETTINGS

- A set of features enabled for the namespace.
- This table consists of a feature category (such as Duo Chat) and a feature (such Code Completion)
- This also stores the Namespace level recommended model
- New
- Why?
 - We need to enable group admins to optionally set the models they want to use for each of the features.

5. NAMESPACEAIFEATUREMODELS

- Stores a list of allowed models for each feature. If this list if empty then we will show all the GitLab supported models to the user
- Future

6. USERS

- Represents individual users who can interact with AI features.
- Existing

7. USERAIMODELPREFERENCES

- Allows users to select their preferred AI model per feature, overriding the namespace default.
- Future

- Why?
 - Empowers users with flexibility while maintaining organizational defaults.
 - Allows for personalization of AI-assisted workflows.

> Note: An organizationid field will be added to AISELFHOSTEDMODELS, AIFEATURESETTINGS and USERAISETTINGS so that those work with cells.

Key Design Considerations

1. Default & Override Mechanism

- Namespace Admins define default AI models for each feature.
- Users can override these defaults for personal preferences (in the future).
- Fallback Logic:
 1. Check USERAISETTINGS for user preference.
 2. If no user preference, check namespace in NAMESPACEFEATURESETTINGS.
Currently we just check the top-level namespace default.
In future iteration we might check for namespace selection cascadingly (from the closest parent to the root ancestor).
 3. If no namespace default, revert to a system-wide default.
- Why?
 - Ensures a structured decision-making process.
 - Gives users autonomy without breaking organizational policies.

2. Scalability & Performance

- We need to be able to hierarchically look up the list of models across the namespace hierarchy.

3. Incident management

- When the default model is selected or when model selection is disabled, an incident management mechanism is being put in place. The mechanism that will be implemented in AI Gateway will re-route the call to another model when the default model can't respond. Related issue.
- When a model has been selected for a feature by the user, we can't perform any incident management due to the fact that the user's choice needs to be respected. The user needs to be aware of this behavior.

4. Flexibility for Deprecation

- We need to be able to think about how administrators can deprecate models and what the process would be to cascade that down to other levels and the database performance implications of that.

Changes to GitLab rails

1. The list of instance-level models is fetched from AI Gateway when a user is interacting with the model selection settings page and cached in the instance for an hour.

1. We need a way to sync models to all cells in .com and all self managed instances. This could be done using a separate sidekiq job that will sync GitLab Managed models with each cell and the self-managed instance.

mermaid

sequenceDiagram

```
Self Managed->>Cloud Connector: Scheduled Sidekiq job calls API
Cloud Connector->>AI Gateway: Fetch Model List
AI Gateway->>AI Gateway: Fetch Model List
AI Gateway-->>Cloud Connector: Return list of models
Cloud Connector-->>Self Managed: Return list of models
Self Managed->>Self Managed: Insert model records and set defaults
Self Managed->>Administrator: Send email about new model availability
```

1. We built the rails models, controllers and views as described in the Data Model section for the first iteration.

1. Every feature when making an API call to AI Gateway needs to be able to pick a model from the list of models allowed for a specific namespace, group, etc. Every feature also needs to be able to look at the default model for the feature.

1. We built the Group Settings page to be able to select namespace defaults from the set offered at the top level namespace. Selecting a list of models for the child namespace if for future consideration.

1. When a model is depreciated / inactivated then we need a way to cascade the deprecations down to the namespace level. We will need to build a Sidekiq job that can do that.

1. In rails when the user is picking a recommendedmodel / defaultmodel at a namespace level they would be able to choose between the GitLab managed models (from the AI gateway config) and from the list of models configured in the self-hosted models screen.

Future changes when we allow users to pick a model:

1. Every feature (chat, code suggestions, code review) needs to be able to display a list of available models on the UI. When the customer selects a model that should be set as the default model.

> For some features such as Duo Code Review where the customer is not actively interacting with the UI, we may only allow the selection of a single model.

1. We will need to build a new API to fetch the list of models that user is allowed to use

gql

```
query {
  aiFeatureSettings {
    nodes {
      feature,
      defaultModel,
```

```

    validModels {
      nodes {
        name
      }
    }
  }
}

```

AI Gateway Changes

The AI Gateway will have to support a new API that will allow rails to fetch the allowed list of models.

The configuration is done with this pair of files:

- The model details configuration file where the model metadata will be configured.
- The unit primitive configuration file where the models are assigned to feature settings.

Both files need to be updated when we need to release or deprecate a model.

The AI Gateway already supports model passing for various APIs such as /v2/chat and /v4/suggestions.

The AI Gateway also already supports prompt versioning for different providers so we can tune the prompt in certain cases.

We will need to test our prompt changes across different model families and versions to ensure that we are backwards compatible with what customers are running.

We should collect metrics across different features to figure out the most popular models so that we can focus our model tuning efforts to those.

Important note: There is currently no way with model selection to roll out a model incrementally in the AI Gateway. Once a model is configured, it will be sent to customers. Keep that in mind when adding a new model and commit changes to these files at the end of your development cycle.

Changes to Duo Workflow

In order to allow Model Switching for Duo Workflow we will need to have to create a new field showing the list of models that are available for the GitLab project. Since GitLab project is required for starting a Duo Workflow, we can use the project to determine the group / sub group information to fetch the list of allowed models. When the user selects a model that model information will need to be added to the parameters passed to Duo Workflow Executor which in turn will pass the model name to Duo Workflow Service. The protobuf contract between the Duo Workflow Service and Executor will need to have a new field called model.

proto

```

message StartWorkflowRequest {
  string clientVersion = 1;
  string workflowID = 2;
  string workflowDefinition = 3;
  string goal = 4;
  string workflowMetadata = 5;
  repeated string clientCapabilities = 6;
  string model = 7;
}

```

In addition to this we will need to be able to update the model factory in Duo Workflow to support different models.

IDE Changes

The IDE must call GitLab to retrieve the list of allowed models for each feature and pass the selected model in the request to AI Gateway or Rails.

Related issue.

1. Model List Update

- The IDE periodically fetches the updated list of available models or listens for a specific GitLab: Update Model List command.
- Changes could also be triggered when the user switches GitLab accounts or when an admin updates model availability.

2. User Preferences

- A settings screen in the IDE lets users select their default model per feature.
- The IDE continues to pass the chosen model in chat and code suggestion requests.

Important note

Currently, when a user is assigned a seat to at least one project with a model selected for completion. The IDE disables the direct connection to AI Gateway code completion calls and goes through the GitLab monolith, which ultimately selects the model to be used according to the user's preferences. The customers should be made aware of this through documentation.

mermaid

sequenceDiagram

alt Model Switching

User->>IDE: Opens VSCode

IDE->>Rails: Fetch List of Models

Rails->>Rails: Lookup namespace of project for model list

loop Every namespace with parent

 Rails->>Rails: Lookup parent namespace of project for model list

end

Rails->>Rails: Lookup instance for model list

Rails->>IDE: Return List of models per feature and default

```

IDE->>IDE: Set default models if not set
IDE->>User: Show list of settings options in IDE
User->>IDE: Pick model to be used for each feature
end
alt Code Suggestions
  User->>IDE: User performs code suggestions request
  IDE->>AIGateway: Send Model in suggestions Request
  AIGateway->>AIGateway: Check whether model is allowed from JWT claims
  AIGateway->>LLM: Get suggestions from model
  LLM-->>AIGateway: Suggestions response
  AIGateway->>AIGateway: Post processing
  AIGateway-->>IDE: Send suggestions response
  IDE->>User: Show suggestions
end
alt Chat
  User->>IDE: User interacts with Chat widget
  IDE->>Rails: Start Chat session
  Rails-->>IDE: Chat Session ID
  IDE->>Rails: Send user message
  Rails->>Rails: Use model in request (check model is allowed)
  Rails->>AIGateway: Make chat request with model
  AIGateway->>LLM: Send chat request
  LLM-->>AIGateway: Chat response
  LLM-->>IDE: Send chat response
  IDE->>User: Show response
end

```

> Note: This is a simplified diagram and does not contain all details such as Authentication/Authorization of the requests

Open Questions / Risks

Conflict Resolution

What happens if multiple namespace admins set conflicting default models at different levels of the hierarchy. Which default should we choose?

> Decision: Configurations on a child namespace to take precedence over the parent namespace.

Future Model Feature Parity

Some models have different capabilities (e.g., Google Gemini 2.0 supports internet search and o1 supports thinking). How do we handle enable such sub-features?

Deprecation & Enforcement

Should we force a hard switch if an AI model is marked as deprecated, or provide a grace period?

SECTION: engineering/architecture/design-documents/autoflow/_index.md

{{< engineering/design-document-header >}}

GitLab AutoFlow allows users to encode workflows of interactions between DevSecOps domain objects and external systems.

This design doc is available in the AutoFlow design-doc project.

Who

<!-- vale gitlab.Spelling = NO -->

Who	Role
Grzegorz Bizon	Distinguished Engineer
Natalia Tepluhina	Principal Engineer
Mikhail Mazurskiy	Staff Engineer
Timo Furrer	Senior Engineer

<!-- vale gitlab.Spelling = YES -->

SECTION: engineering/architecture/design-documents/automated_query_analysis/_index.md

{{< engineering/design-document-header >}}

Problem Summary

Our overarching goal is to improve the reliability and throughput of the GitLab database review process. The current process requires merge request authors to manually provide query plans and raw SQL when introducing new queries or updating existing queries. This is both time consuming and error prone.

We believe we can improve operational efficiency by automatically identifying and analyzing newly introduced SQL queries. This will reduce the risk of human error, leading to improved system stability and an overall reduction in performance regressions.

Our key success metric is a reduction in the number of manual actions required by both code contributors and database reviewers, while maintaining a consistent standard for database related code contributions.

Goals

1. Replace the current process of the author manually obtaining SQL and query plans with an automated process.
1. Decrease the incidence of performance regressions when poorly performing

queries are missed by a manual process.

1. Increase contributor and reviewer efficiency by automating the query testing portion of database review.

Challenges

- Capturing the number of SQL queries generated by an application the size of gitlab-org/gitlab without causing an increase in CI time and/or resources may present a challenge.
- Storing the number of SQL queries generated by an application the size of gitlab-org/gitlab may consume large amounts of database storage.

Opportunity

- Automated test suites already generate a large number of SQL queries, for instance rspec test suites, that can be captured and used to perform automated analysis.
- We already utilize postgres.ai to analyze query performance, and it has an API that will allow us to automate the creation of database clones with realistic production-like data in order to perform automated analysis.
- For customers who do not use something like postgres.ai, but who are connecting to a test database in CI, we would use this connection to generate query plans. The accuracy of these query plans will be affected by how realistic the test data is, and can be improved by seeding the test database with production-like data.
- By storing queries and their query plans, we can tokenize the query plan into plan components, assign a cost and weight, then match those against a machine learning model. We can build this model by generating query plans for queries in our slow query logs, and assign actual cost and weight to their plan components. This will allow us to leverage our corpus of queries and slow query logs to predict the performance of arbitrary query text for other applications and our customers.

Proposal

We plan to automate the process of identifying new and changed database queries, so that contributors and reviewers can more accurately and efficiently assess the database performance impact of a code change.

We will capture queries generated as a side effect of running tests in CI, normalize them, deduplicate them, analyze them using one or more analyzers, and store them with their analyses and other metadata for future retrieval and comparison.

We will post a comment to the originating merge request, containing a summary of the new and changed queries, with links to their analyses, and highlighting any queries that exceed established timing or other performance guidelines.

Design and implementation details

Iteration 1

In the first iteration we will focus on how we capture queries, including normalization, deduplication, and storage. We must consider the performance and resource impacts on CI pipelines during capture, and include things like partitioning and time decay for the information we are storing.

Capturing queries

We will strive to limit the time and resource impacts on our CI pipelines as much as possible. These are some of the options we will consider for capturing queries:

- Instrumenting ActiveRecord in ruby
 - Challenges:
 - Only applies to ruby projects so it would not be applicable to projects like container-registry.
 - Has a non-zero impact on time and resources in CI pipelines (these impacts can be observed in [!111638](#))
 - Opportunities:
 - Simple and straightforward to implement.
 - Allows access to more information (eg. stacktrace and calling locations).
- Connection proxy with logging
 - Challenges:
 - Adds complexity and possible performance overhead.
 - Requires maintaining the code for the proxy.
 - Opportunities:
 - Allows us to customize the capture.
 - Allows us to perform normalization/deduplication at capture time.
- Built-in logging in postgresql
 - Challenges:
 - Require adding a configuration to enable logging.
 - May be difficult to obtain the resulting logs.
 - Opportunities:
 - Doesn't require maintaining any code.
 - Light weight in terms of performance impact.
- Capture from pgstatstatements
 - Challenges:
 - Requires creating the extension in the test database.
 - Requires adding a configuration to set pgstatstatements.max to a value high enough to capture all queries.
 - Consumes shared memory proportional to pgstatstatements.max.
 - Opportunities:
 - Requires minimal code.
 - Simple to obtain the data.
 - Data is already normalized.

We have already built a proof of concept for instrumenting ActiveRecord in ruby in

!111638, so as a first step we will benchmark the other capture methods against it and select the best option.

Storing queries

For the next step of the first iteration we will use the proof of concept in !111638 as well

as any data gathered from testing other capture methods to estimate the number of rows per project, and use the pipeline execution statistics for gitlab-org/gitlab to estimate throughput. These estimates will allow us to evaluate storage mechanisms that are suitable for our purpose.

Some of the storage mechanisms we plan to evaluate are:

- In the ci database in the GitLab database instance
 - Challenges:
 - Places additional strain on this resource for GitLab.com.
 - Opportunities:
 - Allows us to utilize existing authentication and access control in the form of CIJOBTOKEN.
 - Allows us to leverage associations with cibuilds and cpipelines.
 - Simplifies deployment for self-managed.
- In a new decomposed database in the GitLab database instance
 - Challenges:
 - Adds to required development and testing effort.
 - Adds to deployment effort for GitLab.com.
 - Opportunities:
 - Isolates database performance impacts from the existing main and ci database instances.
- In a new external service
 - Challenges:
 - Adds to required development and testing effort.
 - Adds to deployment effort for GitLab.com and for self-managed.
 - Opportunities:
 - Isolates performance impacts from gitlab-org/gitlab.
 - Allows us to iterate faster without impacting the main application.
- In ClickHouse
 - Challenges:
 - Not yet available for self-managed.
 - Opportunities:
 - Isolates database performance impacts from the existing main and ci database instances.

An example database schema for storing queries:

sql

```
CREATE TABLE queries (  
  createdat timestamp with time zone NOT NULL,  
  updatedat timestamp with time zone NOT NULL,  
  id bigint NOT NULL,  
  projectid bigint NOT NULL,
```

```

    analysisid bigint,
    hash text,
    sql text
);
CREATE TABLE pipelinequeries (
    id bigint NOT NULL,
    projectid bigint NOT NULL,
    pipelineid bigint NOT NULL,
    queryid bigint NOT NULL
);
CREATE TABLE analyses (
    createdat timestamp with time zone NOT NULL,
    updatedat timestamp with time zone NOT NULL,
    id bigint NOT NULL,
    projectid bigint NOT NULL,
    queryid bigint NOT NULL,
    buffers int,
    walltime int,
    explain text,
    analysisurl text
);

```

One possible method of partitioning a schema like the above example would be to utilize sub-partitioning.

If we partition by projectid then by some interval of updatedat, and touch the row when we see a query, we can store only queries that the codebase is still executing, and prune partitions that only contain queries the code is no longer generating.

Iteration 2

In the second iteration we plan to identify new and changed queries, and post MR comments containing a summary. We will begin soliciting feedback on the accuracy and utility of the information, and improve or filter it to maximize it's usefulness.

Iteration 3+

In the third and following iterations we plan to automate query analysis using one or more analyzers, store these analyses, and add them to the MR comments. We also intend to re-evaluate our use of the database to store query information, and the API to retrieve it, and potentially move this to an external service.

SECTION: engineering/architecture/design-documents/backup_and_restore/_index.md

{{< engineering/design-document-header >}}

Summary

The Unified Backups project provides a single command-line tool that will handle the application backup and recovery needs of GitLab installations across supported reference architectures. It will be packaged separately from the main GitLab code base to keep it decoupled from specific release versions but will be shipped along with GitLab releases.

This tool will be aware of the nuances of each runtime environment configuration and it will make adaptations to capture and restore data appropriately. It will stand as the primary recommended solution for most customers going forward.

Early development on this tool will focus on providing value to self-hosted customers of GitLab by supporting the variety of installation types and common architectures. For these customers, we will focus on simplifying the disaster recovery process into a common set of recommendations. Additionally, we will work to resolve scalability problems with current backup solutions by supporting the cloud service integration capabilities of large high-usage GitLab instances.

However, we will quickly move into the next phase of the project, which will focus on the specialized needs of GitLab Dedicated. This will provide value to the Dedicated Group in terms of automating and streamlining backup creation and restoration with our standard dedicated architectures on Google and Amazon cloud services.

Background

Disaster Recovery

High-usage business-critical applications are vulnerable to a variety of potentially disastrous incidents. Bad actors may exploit security vulnerabilities to hold business data hostage in ransomware attacks. Software bugs may cause massive unexpected data corruption. Cloud providers may experience unexpected outages that inadvertently cause different types of application data to fall out of sync. This is why organizations that run important high-usage applications generally implement disaster recovery (DR) plans. DR plans allow restoring full operation of the target application regardless of whether there is a major failure or a significant mistake in day to day use.

A reliable application data backup is crucial to any disaster recovery plan. A backup captures the state of application data at a particular point in time so that it may be restored at a later time when performing disaster recovery. The captured data is usually stored as a set of one or more files kept in a location that is distinct from the machine where the application typically runs.

System Administration and Maintenance Purposes

While DR is the primary motivation for creating tooling around backup creation and restoration, we also acknowledge that administrators of self-hosted installations often need to use backup and restoration for configuration and maintenance purposes. The administrator may need to upgrade versions of a crucial component of the system such as PostgreSQL or object storage appliance. In such cases, it is often best to take down their GitLab application temporarily to prevent further updates. Then, the admin may capture a point-in-time backup of their

installation. The admin then performs the necessary changes to the system such as upgrading PostgreSQL to a more recent version. When complete, the captured backup data may be restored to ensure GitLab is close to the same state as it was in prior to the maintenance work. Finally, the site may be brought back online for general use again.

Alternatively, an admin may need to migrate their current GitLab production architecture to new infrastructure. For example, an organization may be running a simple GitLab 1K reference architecture on a single machine the organization manages. However, over time, this organization has usage that has grown beyond the parameters of a simple 1K architecture, and now an admin has been charged with setting up a cloud-based 3K multi-node reference architecture hosted through Amazon Web Services (AWS). This is a common use case for backup and restoration tooling. The admin builds out their 3K deployment on AWS. Then, the admin takes a backup of the running 1K deployment. Finally, the admin restores the backup on the 3K architecture to ensure the organization can seamlessly switch from the small architecture to this new high-usage one.

In this last example, we are indicating how backup tools can facilitate customers to move their GitLab installations to different architectures. However, we need to discuss an important limitation for this use case. We only permit restoring a backup archive on the exact same version number of GitLab upon which the backup was created. We have seen many situations in the past where customers attempt to use GitLab backup tools to transfer their application data between different release versions of GitLab itself. As an example, a customer may have primarily used a version 15.10 installation without managing incremental releases updates over time. Now, the customer wants to jump to a much more recent version like 16.10. That customer may believe they can just use the Rake backup creation task to get a tarball of their older site data, setup a new location with the more recent version of GitLab, and then just use the Rake backup restoration task to populate the newer version installation with the same data. However, our current backup tooling does not support restorations on a different GitLab version than the one under which the backup was created.

Currently, we plan on making the new unified backup tool also uphold this restriction; backup restoration will only take place on the same version of GitLab that created the target backup archive. This is essential to make sure the database schema is exactly in line with what is expected by that version. Otherwise, there may be any degree of bugs and data corruptions that may occur when the code is programmed against a different data schema than what has been restored. We do intend to explore some degree of cross-version restoration support in later iterations of the project.

Current GitLab Backup Offering

Currently GitLab provides recommendations for how to create application backups across different installation types and different hosting architectures. We provide a fairly rudimentary set of tools to create a point-in-time application backup, as well as specialized documentation for how to handle more complex cloud backup situations. You can read more on how GitLab backups work [here](#).

These are based on Rake tasks in the GitLab repository.

Drawbacks of the current approach

1. Current Backup Process Is Not Scalable

1. Backups Do Not Support Cloud-Provider Managed Services

1. Rake Tasks Are Not Suitably Maintainable

1. Invocation Differs Between Installation Types and Architectures: Depending on the installation type, the way you invoke the tasks changes. Depending on which reference architecture and whether it's an on-premise or cloud installation, the required option flags and configurations are different. This duplication and divergence makes the knowledge of running backups difficult to transfer between different installation types or different reference architectures.

1. Manual Restoration for Cloud Services: To leverage certain backup features of cloud providers, such as AWS Backup's continuous, point-in-time backup of object storage, you must currently configure it yourself. Even worse, you must also perform restoration manually. GitLab does not provide any tooling to help automate the process.

Project Goals

With Unified Backups we have the following goals:

- Have a single codebase with all the required logic to backup and restore GitLab Installations
- Create the Unified Backup CLI tool that will:
 - Control backup and restore regardless of architecture or installation type
 - Have a simple interface with self-documented options
 - Have a configuration file where credentials and other stable parameters can be stored
 - Have an optional machine-readable outputs in JSON format
- Support two different Backup formats with a similar UX:
 - Portable Backup
 - Ideal for the smallest reference architectures, 1K and 2K
 - Should work for larger on-premise installations (but may become too slow until depending on nature and quantity of data)
 - Could also be used for cloud-based installations where no managed services is being used
 - Cloud-based Backup
 - Relies solely on the Cloud provider APIs and backup capabilities
 - Support for the major cloud providers like: AWS, GCP and Azure
 - It should not require significant disk-space or permanent storage on the machine/container executing it.
 - It should be simple to extend support for additional cloud providers.
- Support for the following installation types with a consistent UX:
 - Source
 - Linux Package (Omnibus)
 - Docker
 - Kubernetes

Non-Goals

- This is not a solution for users to backup/export their GitLab.com group or project data
- The Unified Backup CLI will not handle lifecycle management:
 - It will not handle backup retention policies
 - It will not handle backup rotation
 - It will not handle backup scheduling
- The Unified Backup CLI will not accept ENV variables as way to:

- Provide options
- Provide configurations
- Provide credentials
- Override settings or configurations from dependent tools
- The Unified Backup CLI will not integrate Deployment / Infrastructure functionality:
 - It will not run GET or Terraform
 - It will not make any change to existing infrastructure
 - It will not deploy new infrastructure

Design and implementation details

Design Decisions

A centralized codebase

By centralizing the codebase into a single project, we aim to simplify the implementation for the permutations of environments and installation types we support. Having everything in a single location makes it easier to test and extend the code.

Supporting multiple cloud providers

Adding support to new cloud providers will follow an approach similar to the adapter pattern, where we have a generic business logic on how to backup each data-type, and a specialized version for each cloud provider.

Installation type based configuration handling

To support the differences in each installation type, a wrapper may be used which provides the tool with the necessary information about the environment it is running on.

Differentiating between Portable and Cloud Backups

The clear separation between Portable and Cloud backups aims at simplifying the implementation, and allows for specific strategies to be explored in the future to take consistent backups without downtime.

The main difference between the two is that Portable backups are stored locally. Both styles of backups allow mixed data storage, e.g. application data split between local and cloud storage.

Consistency in backups

Consistent backups can be taken during downtime.

Online Consistent Backups will be explored through epic 12043.

On-going design discussions

The results of the following on-going technical design discussions will be added to the blueprint.

- Investigate scaling backups
- How to limit concurrency of backup processes?
- How to restore a cloud backup with the gitlab-backup-cli?
- Add how distribution will work with this tool
- Investigate how to handle mixed Portable and Cloud backups
- Investigate what is required to support Kubernetes and large reference architectures
- Investigate how to support backing up Omnibus configuration and secrets
- What infra provisioning is needed for each type of datatype being backed up in the cloud?
- How to handle Redis data (not backed up)?
- Investigate Gitlab improvements to Backup

Limitations

- We don't support the data in a Cloud-based Backup to be exportable to a Portable Backup format or vice-versa.
- We do not support backing up data in the cache store (Redis) which includes the Sidekiq state. TODO: More research on Redis stored data

Backup types

We provide two different approaches to create a Backup: Portable and Cloud-based.

Portable Backup

A Portable Backup behaves similar to the existing Rake-task approach. It relies on direct access to data-types and dependent services. It is the most compatible solution, as it does not rely on Cloud specific functionality.

A Portable Backup could be executed in all supported installation types and works on both physical and virtual machines.

With Portable Backup, the data is transferred to the machine running the backup context, stored and compressed in a specific location. A metadata file is created and the final archive can be stored locally or in a remote location (including in an Object Storage Bucket).

Each data type can be backed up in sequence or in parallel. Similarly, each one can be restored in sequence or in parallel.

Multiple Backups can be executed simultaneously, without one overwriting the other. (This is intended to support executing a backup from a cron job)

Backing up the database works the same, and require no additional configuration, no matter if using Omnibus-managed database, Patroni, an externally managed instance or whether PgBouncer is enabled.

Future work will look into improving the consistency of Portable Backups without downtime, which may rely on using specific file-systems that can provide snapshotting capabilities.

Portable Backups have inherent limitations in scale, depending on the infrastructure, the data, and DR needs.

Cloud-based Backup

A Cloud-based Backup relies entirely on a Cloud Provider specific APIs and Services to perform and store the data. It is triggered by the CLI tool which orchestrates the required API calls.

Each time a Backup is started, a JSON file containing the necessary information (Backup Metadata) to find each data type is stored in the Backup Bucket, under the same prefix used to store existing Blobs.

The Backup Metadata will be used as the SSOT to specify what data is part of a Backup. In order to list or process a Backup, the tool will fetch the JSON files from the Backup Bucket. Based on the IDS and references stored there, the tool will be able to execute the restore-actions in the Cloud Provider API.

As we rely on the Backup Metadata, we primarily support listing and restoring data that was created by triggering the Unified Backup tool. This gives us the control and the foundation to build the necessary features and improvements for a Consistent Backup.

The tool will not rely on Cloud Provider specific scheduled backups functionality. To achieve regular scheduled backups, the tool will need to be invoked by an external service like a Cronjob.

Multiple backups may be supported to run simultaneously (unless there is a specific Cloud Provider limitation). A backup could include all supported data-types or specific ones only.

For each supported Cloud Provider, specific capabilities may be available, but we will support at least the following for each data-type:

- Databases:
 - On-Demand Database Backup or Snapshot
- Blobs in Object Storage
 - A copy of all Blobs in a new Backup Bucket
- Git repositories
 - A disk snapshot for all running Gitaly instances (Gitaly Cluster is not supported)

As Gitaly releases its planned new backup related functionality we will evaluate integrating them.

See [future-iterations] for information regarding scheduling backups.

Google Cloud Platform

For GitLab installations hosted on Google Cloud Platform (GCP) we will support the following application data types and their corresponding backup mechanism:

Data Type	GCP Service Component	GCP backup mechanism
-----	-----	-----

| PostgreSQL Databases | Cloud SQL for PostgreSQL | On-demand database snapshots
|
| Blob/Files | Cloud Storage (Object Storage) | On-demand regional data
transfer using Storage Transfer Service |
| Repositories | Compute Engine with Persistent disks | On-demand disk snapshots
|

As we proceed with the implementation we will consider implementation details on how to use their APIs, including the differences between synchronous and asynchronous operations, support for Batching / Parallelization, Cloud Logging the use of Pub/Sub notifications, etc.

Cloud SQL for Postgres (Backup)

Cloud SQL supports the following methods to perform a Backup:

- Automated (scheduled)
 - Can be scheduled to a specific starting time (execution happens with a 4 hour window)
 - Retention of automated backups:
 - Cloud SQL Enterprise edition: Minimum 7 units (usually days) max: 365 units
 - Cloud SQL Enterprise Plus edition: Minimum 15 units (usually days) max: 365 units
 - Retention of WAL logs
 - Cloud SQL Enterprise edition: From 1 to 7 days (default: 7 days)
 - Cloud SQL Enterprise Plus edition: From 1 to 35 days (default: 35 days)
 - WAL logs can be stored on disk in the same instance or in Cloud Storage
- On-Demand Backups
 - Are not automatically deleted (can be stored for long time)

We will provide initial Backup support relying on On-Demand Backups to support long-term retention periods.

During the implementation phase we will investigate how to support and expose additional options like:

- Ability to choose a custom region to store Backups (default: same region as the instance)
- Ability to customize Automated Backup retention policy (normally managed by Terraform or as part of instance configuration)

Cloud SQL for Postgres (Restore)

Initial Restore support will be based on On-Demand Backups. We will rely on the IDs stored as part of the Backup Metadata file read from the Backup Bucket.

During the implementation phase we will investigate how to support and expose additional options like:

- Ability to use Point In Time Recovery based on Automated Backups WAL logs (when in range)
- Restore Backup in a different instance then the current being used (we will not handle instance provisioning)
- How to expose the information necessary for data be restored as part of an external infrastructure integration (E.g. provide data for a Terraform script to create a new

deployment)

Blobs

Backup

GCP backups currently use the Storage Transfer Service.

Storage Transfer Service jobs are created that copy from the individual buckets (Uploads, LFS, etc) to one bucket, under the path /backups/\$BACKUPID/\$TARGET.

Restore

- Restores should create Storage Transfer Service jobs that copy from the backup bucket path, to the individual buckets.
- We will not create or configure the buckets, that is out of scope for the tool.
- We (probably) should not empty the buckets first. Leave that up to the users to do if they want to.
- So at its most basic, a user would run something like `gitlab-backup-cli restore all $BACKUPID`, and the tool would create the necessary jobs to copy data from the backup bucket path, to the individual buckets, then monitor them for success/failure.

Integrating with the Unified Backup CLI

In order to make the tool easy to integrate with external tools, we will provide optional machine-readable output in JSON format.

As an example, in order to restore from a disk snapshot, as the tool will not handle that operation itself, we will provide a command that can list the resources that should be restored in JSON format. That can be read by any external integration tool.

This type of interface is intended to decouple the Backup CLI from any other specific tool.

Distribution

- Omnibus
- Kubernetes

We will work on Omnibus before Kubernetes for a quicker first iteration.

Impact

- Currently, the Distribution team owns and maintains backup-utility which is used for backups in Kubernetes enabled environments. We will need to work with them to replace that tool, with `gitlab-backup-cli`
- For the omnibus-gitlab, it does create the `gitlab-backup` wrapper around the backup Rake tasks. Deprecation of the previous tool is TBD.
- The new tool (`gitlab-backup-cli`) will be distributed as part of the main GitLab codebase as one of the bundled gems.
- In the future, the new tool will be decouple from that codebase and may be distributed independently from the main package, to allow for supporting backing up and restoring from

distinct (compatible) GitLab versions.

Stable Counterpart

Robert Marshall

Dedicated/Cells Deployment

As we proceed with the implementation and initial release, we will work together with internal teams to ensure the tool provides the necessary machine-readable information necessary to integrate with their existing tools.

Specific to Kubernetes we will explore how to integrate with its native Cronjob functionality to provide scheduled executions.

1st Milestone: Create backup cli: support 1K Linux package reference architecture with local storage

See epic

- Targeting Linux package installations
- Behaves similar to the existing backup Rake tool, with a small feature-set

2nd Milestone: Implement Cloud Backups: support 10K CNH reference architectures on GCP

See epic

For the initial Cloud Backup implementation:

- Google Cloud Provider only (other Cloud Providers will be added at a later stage)
 - Database Backups using Cloud SQL Backups (on demand backups only, initially)
 - Object Storage Backups using Storage Transfer Service
 - GCE disk snapshots initially for repository backups.
 - We will revisit GitLab server side backups with WAL partition archives when the technology has matured.
- Only support data/snapshots managed by the Backup tool
- Not relying on automated/scheduled Backup implementation (like AWS Backup or Google Cloud Backup)

In this iteration, we are NOT aiming to solve backup consistency:

- When backing up the multiple components, what is in the database may point to something that was removed before it could have been included in the backup
- The Cloud Providers may not provide a reliable way to match each data to a specific point-in-time (that could be used to synchronize with the database snapshot)

3rd Milestone: Extend support to 25+K reference architectures

See epic

TBD

Future iterations

Investigate integration with continuous database backup (WAL log based) with a rollback to point-in-time as a Restore mechanism. Each cloud provide may implement this in different ways, with different retention policies.

We haven't planned yet any integration with AWS Backup or equivalent, but this is something we will revisit in the future.

Improvements to the Portable Backup:

- Scalable Database solution
- Encryption at Rest (relying on .7z format)
- File-system snapshot support
- Integrated GitLab configuration files and on-disk secrets in separate backup archive

Backup Management solution, that will be composed of:

- A Web UI (decoupled from the main application)
- A REST API to control and manage Backup/Restore related operations
- Observability over Disaster Recovery operation
- Additional Service Orchestration (possible integration with GET)
- Backup Lifecycle management (as e.g.):
 - Backup retention policies
 - Backup execution policies
 - Optional granularity for different types of data
 - Notifications
- Support for both Portable and Cloud-Based backups

SECTION: engineering/architecture/design-documents/backup_and_restore/glossary.md

archive

- The directory or archive file that is produced by a backup creation command. It will always contain a backup metadata file, and it may or may not contain actual captured data from the application, depending on the software environment

archive metadata

- The record of a backup creation session stored in a JSON file inside every archive created by the gitlab-backup-cli command

backup home

- The base directory that a user has configured as the default location to store archives produced by backup creation commands. This should always be treated as distinct from any actual

archive directory or other significant path on the system.

backup context

- When you execute a backup creation or restoration operation, you are running a backup session. It encompasses the operation, command line arguments, and the parameters of the operation as derived from the system configuration, command line arguments, environment variable, application context data, and archive metadata.

data type

- One of the five abstract categories of data used by the GitLab application · Git repositories, databases, blobs, secrets/configurations, and transient data.

- This is distinct from the individual feature data backups contained in an archive. For example, ·artifacts· or ·packages· are two types of blob-based file backups in an archive, but they are not data families. ·Blob· is the general data family to which these feature backups belong.

installation types

- An installation type is the result of using one of the installation methods.

SECTION: engineering/architecture/design-documents/bundle_uri/_index.md

{{< engineering/design-document-header >}}

Summary

bundle-URI is a fairly new concept in Git that allows the client to download one or more bundles in order to bootstrap the object database in advance of fetching the remaining objects from a remote. By having the client download static files from a simple HTTP(S) server in advance, the work that needs to be done on the remote side is reduced.

Git bundles are files that store a packfile along with some extra metadata, including a set of refs and a (possibly empty) set of necessary commits. When a user clones a repository, the server can advertise one or more URIs that serve these bundles. The client can download these to populate the Git object database. After it has done this, the negotiation process between server and client start to see which objects need be fetched. When the client pre-populated the database with some data from the bundles, the negotiation and transfer of objects from the server is reduced, putting less load on the server's CPU.

Motivation

When a user pushes changes, it usually kicks off a CI pipeline with a bunch of jobs. When the CI runners all clone the repository from scratch,

if they use git clone,
they all start negotiating with the server what they need to clone. This is
really CPU intensive for the server.

Some time ago we've introduced the
pack-objects,
but it has some pitfalls. When the tip of a branch changes, a new packfile needs
to be calculated, and the cache needs to be refreshed.

Git bundles are more flexible. It's not a big issue if the bundle doesn't have
all the most recent objects. When it contains a fairly recent state, but is
missing the latest refs, the client (that is, the CI runner) will do a "catch up" and
fetch additional objects after applying the bundle. The set of objects it has to
fetch from will Gitaly be a lot smaller.

Goals

Reduce the work that needs to be done on the Gitaly servers when a client clones
a repository. This is particularly useful for CI build farms, which generate a
lot of traffic on each commit that's pushed to the server.

With the use bundles, the server has to craft a smaller delta packfiles
compared to the pack files that contain all the objects when no bundles are
used. This reduces the load on the CPU of the server. This has a benefit on the
packfile cache as well, because now the packfiles are smaller and faster to
generate, reducing the chances on cache misses.

Non-Goals

Using bundle-URIs will not reduce the size of repositories stored on disk.
This feature will not be used to offload repositories, neither fully nor
partially, from the Gitaly node to some cloud storage. In contrary, because
bundles are stored elsewhere, some data is duplicated, and will cause increased
storage costs.

In this phase it's not the goal to boost performance for incremental
fetches. When the client has already cloned the repository, bundles won't be
used to optimize fetches new data.

Currently bundle-URI is not fully compatible with shallow clones, therefore
we'll leave that out of scope. More info about that in
Git issue 170.

Proposal

When a client clones a repository, Gitaly advertises a bundle URI. This URI
points to a bundle that's refreshed on a regular interval, for example during
housekeeping. For each repository only one bundle will exist, so when a new one
is created, the old one is invalidated.

The bundles will be stored on a cloud Object Storage. To use bundles, the administrator should configure this in Gitaly.

Design and implementation details

When a client initiates a git clone, on the server-side Gitaly spawns a git upload-pack process. Gitaly can pass along additional Git configuration. To make git upload-pack advertise bundle URIs, it should pass the following configuration:

- uploadpack.advertiseBundleURIs :: This should be set to true to enable to use of advertised bundles.
- bundle.version :: At the moment only 1 is accepted.
- bundle.mode :: This can be either any or all. Since we only want to use bundles for the initial clone, any is advised.
- bundle.<id>.uri :: This is the actual URI of the bundle identified with <id>. Initially we will only have one bundle per repository.

Enable the use of advertised bundles on the client-side

The current version of Git does not use the advertised bundles by default when cloning or fetching from a remote.

Luckily, we control most of the CI runners ourself. So to use bundle URI, we can modify the Git configuration used by the runners and set `transfer.bundleURI=true`.

Access control

We don't want to leak data from private repositories through public HTTP(S) hosts. There are a few options for how we can overcome this:

- Only activate the use of bundle-URI on public repositories.
- Use a solution like signed-URLs.

Public repositories only

Gitaly itself does not know if a project, and its repository, is public, so to determine whether bundles can be used, GitLab Rails has to tell Gitaly. It's complex to pass this information to Gitaly, and using this approach will make the feature only available for public projects, so we will not proceed with this solution.

Signed URLs

The use of signed-URLs is another option to control access to the bundles. This feature, provided by Google Cloud, allows Gitaly to create a URI that has a short lifetime.

The downside to this approach is it depends on a feature that is cloud-specific, so each cloud provider might provide such feature slightly

different, or not have it. But we want to roll this feature out on GitLab.com first, which is hosted on Google Cloud, so for a first iteration we will use this.

Bundle creation

Use server-side backups

At the moment Gitaly knows how to back up repositories into bundles onto cloud storage. The documentation describes how to use it.

For the initial implementation of bundle-URI we can piggy-back onto this feature. An admin should create backups for the repositories they want to use bundle-URI. With the existing configuration for backups, Gitaly can access cloud storage.

As part of housekeeping

Gitaly has a housekeeping worker that daily looks for repositories to optimize. Ideally we create a bundle right after the housekeeping (that is, garbage collection and repacking) is done. This ensures the most optimal bundle file.

There are a few things to keep in mind when automatically creating bundles:

- Does the bundle need to be recreated? When there wasn't much activity on the repository it's probably not needed to create a new bundle file, as the client can fetch missing object directly from Gitaly anyway. The housekeeping tasks uses various heuristics to determine which strategy is taken for the housekeeping job, we can reuse parts of this logic in the creation of bundles.
- Is it even needed to create a bundle? Some repositories might be very small, or see very little activity. Creating a bundle for these, and duplicating it's data to object storage doesn't provide much value and only generates cost and maintenance.

Controlled by GitLab Rails

Because bundles increase the cost on storage, we eventually want to give the GitLab administrator full control over the creation of bundles. To achieve this, bundle-URI settings will be available on the GitLab admin interface. Here the admin can configure per project which have bundle-URI enabled.

Configuration

To use this feature, Gitaly needs to be configured. For this we'll add the following settings to Gitaly's configuration file:

- `bundleuri.strategy` :: This indicates which strategy should be used to create and serve bundle-URIs. At the moment the only supported value is "backups". When this setting to that value, Gitaly checks if a server-side

backup is available and use that.

- `bundleuri.signurls` :: When set to true, the cloud storage URLs are not passed to the client as-is, but are transformed into a signed URL. This setting is optional and only support Google Cloud Storage (for now).

The credentials to access cloud storage are reused as described in the Gitaly Backups documentation.

Storing metadata

For now all metadata needed to store bundles on the cloud is managed by Gitaly server-side backups.

Bundle cleanup

At some point the admin might decide to cleanup bundles for one or more repositories, an admin command should be added for this. Because we're now only using bundles created by gitaly-backup, we leave this out of scope.

Gitaly Cluster compatibility

Creating server-side backups doesn't happen through Praefect at the moment. It's up to the admin to address the nodes where they want to create backups from. If they make sure the node is up-to-date, all nodes will have access to up-to-date bundles and can pass proper bundle-URI parameters to the client. So no extra work is needed to reuse server-side backup bundles with bundle-URI.

Alternative Solutions

No alternative solutions are suggested at the moment.

SECTION: engineering/architecture/design-documents/capacity_planning/_index.md

{{< engineering/design-document-header >}}

Summary

This document outlines how we plan to set up infrastructure capacity planning for GitLab Dedicated tenant environments, which started as a FY24-Q3 OKR.

We make use of Tamland, a tool we build to provide saturation forecasting insights for GitLab.com infrastructure resources.

We propose to include Tamland as a part of the GitLab Dedicated stack and execute forecasting from within the tenant environments.

Tamland predicts SLO violations and their respective dates, which need to be reviewed and acted upon.

In terms of team organisation, the Dedicated team is proposed to own the tenant-side setup for

Tamland and to own the predicted SLO violations, with the help and guidance of the Scalability::Observability team, which drives further development, documentation and overall guidance for capacity planning, including for Dedicated.

With this setup, we aim to turn Tamland into a more generic tool, which can be used in various environments including but not limited to Dedicated tenants.
Long-term, we think of including Tamland in self-managed installations and think of Tamland as a candidate for open source release.

Motivation

Background: Capacity planning for GitLab.com

Tamland is an infrastructure resource forecasting project owned by the Observability team. It implements capacity planning for GitLab.com, which is a controlled activity covered by SOC 2.

As of today, it is used exclusively for GitLab.com to predict upcoming SLO violations across hundreds of monitored infrastructure components.

Tamland produces a report (internal link, hosted on GitLab Pages) containing forecast plots, information around predicted violations and other information around the components monitored. Any predicted SLO violation results in a capacity warning issue being created in the issue tracker for capacity planning on GitLab.com.

At present, Tamland is quite tailor made and specific for GitLab.com:

1. GitLab.com specific parameters and assumptions are built into Tamland
1. We execute Tamland from a single CI project, exclusively for GitLab.com

Turning Tamland into a tool we can use more generically and making it independent of GitLab.com specifics is subject of ongoing work.

For illustration, we can see a saturation forecast plot below for the disk space resource for a PostgreSQL service called patroni-ci.

Within the 90 days forecast horizon, we predict a violation of the soft SLO (set at 85% saturation) and this resulted in the creation of a capacity planning issue for further review and potential actions.

At present, the Scalability::Observability group reviews those issues and engages with the respective DRI for the service in question to remedy a saturation concern.

 />

For GitLab.com capacity planning, we operate Tamland from a scheduled CI pipeline with access to the central Mimir, which provides saturation and utilization metrics for GitLab.com.

The CI pipeline produces the desired report, exposes it on GitLab Pages and also creates capacity planning issues.

Scalability::Observability runs a capacity planning triage rotation which entails reviewing and prioritizing any open issues and their respective saturation concerns.

Problem Statement

With the number of GitLab Dedicated deployments increasing, we need to establish capacity planning processes for Dedicated tenants.

This is going to help us notice any pending resource constraints soon enough to be able to upgrade the infrastructure for a given tenant before the resource saturates and causes an incident.

Each Dedicated tenant is an isolated GitLab environment, with a full set of metrics monitored. These metrics are standardized in the metrics catalog and on top of these, we have defined saturation metrics along with respective SLOs.

In order to provide capacity planning and forecasts for saturation metrics for each tenant, we'd like to get Tamland set up for GitLab Dedicated.

While Tamland is developed by the Scalability::Observability and this team also owns the capacity planning process for GitLab.com, they don't have access to any of the Dedicated infrastructure as we have strong isolation implemented for Dedicated environments. As such, the technical design choices are going to affect how those teams interact and vice versa. We include this consideration into this documentation as we think the organisational aspect is a crucial part of it.

Key questions

- 1. How does Tamland access Prometheus data for each tenant?
- 1. Where does Tamland execute and how do we scale that?
- 1. Where do we store resulting forecasting data?
- 1. How do we consume the forecasts?

Goals: Iteration 0

- 1. Tamland is flexible enough to forecast saturation events for a Dedicated tenant and for GitLab.com separately
- 1. Forecasting is executed at least weekly, for each Dedicated tenant
- 1. Tamland's output is forecasting data only (plots, SLO violation dates, etc. - no report, no issue management - see below)
- 1. Tamland stores the output data in a S3 bucket for further inspection

Goals: Iteration 1

In Iteration 0, we've integrated Tamland into GitLab Dedicated environments and started to generate forecasting data for each tenant regularly.

In order to consume this data and make it actionable, this iteration is about providing reporting functionality for GitLab Dedicated:

We generate a GitLab Pages deployed static site that contains individual Tamland reports for all tenants.

We use the default Tamland report to generate the per-tenant report.

In a future iteration, we may want to provide another type of report specifically tailored for

GitLab Dedicated needs.

Goals: Iteration 2

In order to raise awareness for a predicted SLO violation, Tamland has functionality to manage a GitLab issue tracker and create an issue for a capacity warning.

We use this, for example, to manage capacity warnings for GitLab.com using the gitlab-com capacity planning tracker.

For GitLab Dedicated tenants, we suggest to use the gitlab-dedicated capacity planning tracker in a similar fashion:

For each predicted SLO violation with reasonable confidence, we create a capacity warning issue on this tracker and use a scoped label to distinguish warnings for different tenants (see below for more details).

Non-goals

Customizing forecasting models

Forecasting models can and should be tuned and informed with domain knowledge to produce accurate forecasts.

This information is a part of the Tamland manifest.

In the first iteration, we don't support per-tenant customization, but this can be added later.

Proposed Design for Dedicated: A part of the Dedicated stack

Dedicated environments are fully isolated and run their own Prometheus instance to capture metrics, including saturation metrics.

Tamland will run from each individual Dedicated tenant environment, consume metrics from Prometheus and store the resulting data in S3.

From there, we consume forecast data and act on it.

Generating forecasts

Storage for output and cache

Any data Tamland relies on is stored in a S3 bucket.

We use one bucket per tenant to clearly separate data between tenants.

1. Resulting forecast data and other outputs
1. Tamland's internal cache for Prometheus metrics data

There is no need for a persistent state across Tamland runs aside from the S3 bucket.

Benefits of executing inside tenant environments

Each Tamland run for a single environment (tenant) can take a few hours to execute.

With the number of tenants expected to increase significantly, we need to consider scaling the

execution environment for Tamland.

In this design, Tamland becomes a part of the Dedicated stack and a component of the individual tenant environment.

As such, scaling the execution environment for Tamland is solved by design, because tenant forecasts execute inherently parallel in their respective environments.

Distribution model: Docker + Helm chart

Tamland is released as a Docker image along with a Helm chart, see Tamland's README for further details.

Tamland Manifest

The manifest contains information about which saturation metrics to forecast on (see this [manifest example for GitLab.com](#)).

This will be generated from the metrics catalog and will be the same for all tenants for starters.

In order to generate the manifest from the metrics catalog, we setup dedicated GitLab project tamland-dedicated.

On a regular basis, a scheduled pipeline grabs the metrics catalog, generates the JSON manifest from it and commits this to the project.

On the Dedicated tenants, we download the latest version of